Individual reflection: Median of medians (BFPRT) (Michael Lunn)

1. What is the worst case, and what triggers it?

In theory, BFPRT has no bad input. Every round discards at least 30% of the range whatever the data looks like, so its worst case is O(n) on every input. The pivot depends on the medians of every group of five, not on a few fixed positions, so sorted, organ pipe and duplicate-heavy inputs have nothing to line up with. In testing I found no input that made it clearly slower than random input. The closest to a bad case was two distinct values, at 1.18× comparisons, and even then, its time was 0.81× random.

BFPRT pays for this guarantee on every input by sorting groups and a recursive call before any partitioning. So in practice its weakness is not a trigger input but the constant overhead on ordinary data.

2. Hardest bug or design decision

My implementation was initially written for integers only, but the string experiment needed it to work on any element type. I made it generic over the element type and the comparator, so every comparison goes through the less function. That also let the counting comparator see every comparison. This was tied to a wider problem we found later on which was the first benchmark used its own copies of the algorithms, so it could pass while our code was broken. I only caught it by deliberately breaking my algorithm and seeing the benchmark still pass. We then rewired the harness to call the registered, generic implementations.

3. What would I do differently?

I would pay closer attention to the std::nth_element implementations before running anything. I meant to use libstdc++'s implementation of std::nth_element which to my understanding was an optimised version of introselect, with a depth limit and a heap-select fallback. My MacBook's Apple clang uses libc++, which, as far as we can tell, is a median-of-3 quickselect with no fallback. I only found this out when organ pipe produced 15.6× the comparisons and I had to work out why. Next time I would check which standard library the toolchain uses, read both sources, benchmark both, and describe them correctly in the report from the start.

4. What did I not understand before, and now do?

I did not fully understand why the median of medians guarantees that at least 30% of elements are removed. I could gather that the actual process of finding the median of medians would produce a good pivot but didn't quite understand the exact math behind it.

While understanding this specific point was not strictly necessary for the implementation, I did do further research to gain a better understanding of why this was the case as it is the basis of why the algorithm can guarantee a linear worst case.